

Hamming Distance Report

Explanation for Code:

For the code I decided on using XOR to compare different bits of two strings and then measure the number of differences in the bit by comparing each character from the two strings using XOR. I also used For loops to iterate through each character then compare them to get the differing bits and then iterate to the next character for each word until there are no more characters. In my code section .data initializes all the variables including the length of each word which is 3. In section .text I move the strings into esi and edi registers and let ecx contain the length of the two words. I also use edx as a counter for adding up Hamming distances. Inside the for_loop we move the first character into al and bl for both words and then xor them to find out how many characters differ and we use ebx to count the number of differing bits when two characters are compared. The if_bit_count tests to see if the first bit is 1 in al if it isn't then we jump to else_if_bit_count and if it is then we increment ebx (stores number of differing bits). In else_if_bit_count since we got 0 we will shift right to the next bit and add ebx to edx to properly reflect Hamming Distance and then increment edi and esi so that we move to the next character for comparison and decrement ecx since now the length of the words will be 2 instead of 3 and we keep doing this until length is 0. We jump back to the for_loop to continue this process.

Code:

```
SECTION .text
global _start

_start:

section .data
    str1 db "foo", 0
    str2 db "bar", 0
    len equ 3
    msg db "Hamming Distance: ", 0
    msg_len equ 19
    newline db 10
section .bss
    hamming resb 1

section .text
    global _start

_start:

    mov esi, str1
    mov edi, str2
```

```
mov ecx, len
mov edx, 0 ;used to add up Hamming distance
```

```
for_loop:
    mov al, [esi]
    mov bl, [edi]
    xor al, bl
```

```
    mov ebx, 0 ;used for counting differing bits
```

```
if_bit_count:
    test al, 1
    jz else_if_bit_count
    inc ebx
```

```
else_if_bit_count:
    shr al, 1
    jnz if_bit_count
```

```
    add edx, ebx
```

```
    inc esi
    inc edi
    dec ecx
    jnz for_loop
```

```
    mov eax, edx
    add eax, '0'
    mov [hamming], al
```

```
    mov eax, 4
    mov ebx, 1
    mov ecx, msg
    mov edx, msg_len
    int 0x80
```

```
    mov eax, 4
    mov ebx, 1
    mov ecx, hamming
    mov edx, 1
    int 0x80
```

```
mov eax, 4
mov ebx, 1
mov ecx, newline
mov edx, 1
int 0x80
```

```
mov eax, 1
xor ebx, ebx
int 0x80
```

Output:

```
base login: wed Mar  5 16:38:07 2023 from 198.51.99.145
[neevp1@linux6 ~]$ nasm -f elf32 foo.asm
[neevp1@linux6 ~]$ nasm -f elf32 foo.asm
[neevp1@linux6 ~]$ ld -m elf_i386 -s -o foo foo.o
[neevp1@linux6 ~]$ ./foo
Hamming Distance: 8
```